

ResumeIQ: Technical Reference & NLP Guide

A structured overview of project architecture, source file roles, applied NLP concepts, and execution flow.

1. File-by-File Breakdown: Which File Does What?

Backend Architecture (/backend)

File Name	Role & Core Responsibility
main.py	FastAPI Entry Point & Router: Coordinates endpoints (/api/analyze, /api/health), handles CORS policies, validates uploads, and orchestrates the NLP pipeline.
services/pdf_extractor.py	PDF Text Ingestion: Uses PyMuPDF (fitz) to extract text streams across all pages from uploaded binary bytes. Handles password encryption and scanned PDF errors.
utils/text_preprocessor.py	NLP Preprocessing Engine: Normalizes whitespace, cleans unicode bullets, performs word tokenization via NLTK, removes English stopwords, and supports lemmatization.
services/resume_parser.py	Information Extraction (IE): Uses regex rules & title-case heuristics to detect Name, Email, Phone, and segments sections (Education, Experience, Projects, Certifications).
services/skill_extractor.py	Controlled Lexicon Matcher: Matches text against skills.json using word boundaries (\b), normalizes aliases (<i>JS</i> → <i>JavaScript</i> , <i>NodeJS</i> → <i>Node.js</i>), and prevents false matches.
services/jd_parser.py	Job Description Parser: Cleans pasted JD requirements, counts words, and extracts demanded technical skills using the controlled skill dictionary.
services/similarity.py	Set Comparison Engine: Applies set algebra to calculate Matching Skills (intersection), Missing Skills, Additional Skills, and the Skill Overlap percentage.
data/skills.json	Skill Knowledge Base: Structured JSON dictionary defining canonical skills categorized by domain (Languages, Web, Databases, Cloud, Tools) with their known aliases.
Procfile	Deployment Script: Specifies the cloud start command for Render (uvicorn main:app --host 0.0.0.0 --port \$PORT).

Frontend Architecture (/frontend)

File Name	Role & Core Responsibility
src/App.jsx	Main Dashboard View: Holds state (resumeFile, jobDescription, result, isLoading), handles upload triggers, and coordinates component rendering.
src/services/api.js	Backend Communication: Handles fetch POST requests with FormData. Normalizes endpoint URLs, manages timeouts, and provides diagnostic error messages.
src/components/UploadSection.jsx	User Input Area: Drag-and-drop PDF dropzone with size validation, large Job Description textarea with word counter, and one-click demo data loader.
src/components/SummaryCards.jsx	High-Level Metrics: Displays top cards for Document Name, Total Skills Detected, Matching Skills Count, and Skill Overlap percentage.
src/components/SkillComparisonSection.jsx	3-Way Skill Breakdown: Displays color-coded interactive badge chips: Matching Skills (green), Missing Skills (amber), and Additional Resume Skills (blue).
src/components/ResumeInfoCard.jsx	Candidate Profile Display: Formats extracted contact details (Name, Email, Phone) and standard sections (Education, Experience, Projects). Displays <i>Not detected</i> if missing.
src/components/Header.jsx	App Navigation: Compact header bar displaying application branding and title.
src/index.css	Design System: Custom responsive CSS with modern typography (Inter & Outfit), card shadows, glassmorphism, and color-coded status badges.

2. NLP Concepts Applied & Code Mapping

Every NLP technique implemented in the codebase, its theoretical purpose, and the exact source file location:

NLP Concept	Source File Location	How It Works in This Project
1. Text Ingestion & Extraction	backend/services/pdf_extractor.py	Converts raw binary PDF streams into unformatted plain text page-by-page using PyMuPDF (fitz). Validates against password-protected or scanned image files.
2. Text Normalization & Cleaning	backend/utlis/text_preprocessor.py	Removes formatting noise: converts non-standard unicode bullets and dashes to standard characters, strips non-printable ASCII, and collapses redundant whitespace.
3. Word Tokenization	backend/utlis/text_preprocessor.py	Splits continuous text into word tokens using NLTK's word_tokenize with regex patterns that protect technical symbols like C++, C#, and .NET.
4. Stopword Removal	backend/utlis/text_preprocessor.py	Filters out non-informative English function words (<i>is, at, which, for, the</i>) using NLTK's English stopwords corpus to eliminate false positives in skill lookups.
5. Lemmatization	backend/utlis/text_preprocessor.py	Reduces words to their base dictionary lemma using NLTK's WordNetLemmatizer (e.g. <i>developed</i> → <i>develop</i> , <i>applications</i> → <i>application</i>).
6. Information Extraction (IE)	backend/services/resume_parser.py	Rule-based entity recognition: RFC-standard regex for Email, multi-format phone regex, title-case capitalized heuristics for Name, and boundary detection for resume sections.
7. Controlled Lexicon Matching	backend/services/skill_extractor.py data/skills.json	Matches technical skills against a curated taxonomy using regex word boundaries (\b) and alias resolution (e.g. <i>JS</i> → <i>JavaScript</i> , <i>NodeJS</i> → <i>Node.js</i>) to eliminate hallucinations.
8. Set-Algebra Overlap Analysis	backend/services/similarity.py	Applies mathematical set theory: Intersection ($\text{Resume} \cap \text{JD}$) for Matching Skills; Differences ($\text{JD} \setminus \text{Resume}$, $\text{Resume} \setminus \text{JD}$) for Missing & Additional; and Overlap %.

3. End-to-End Program Flow & Execution Pipeline

How data moves from the user interface through the NLP backend and back to the dashboard:

Step	Pipeline Stage	Technical Execution Details
Step 1	User Input & Upload (Frontend)	The user drops a PDF resume and pastes Job Description text. The React application validates the file type (.pdf only) and ensures minimum required length.
Step 2	API Transmission (FormData POST)	The client constructs a multipart/form-data payload and sends it to the FastAPI endpoint POST /api/analyze. The URL is normalized to support both local and cloud hosts.
Step 3	PDF Stream Ingestion (PyMuPDF)	pdf_extractor.py reads binary bytes from memory (no disk clutter), loads document pages, extracts plain text stream lines, and checks for valid extractable text.
Step 4	Text Preprocessing & Normalization	text_preprocessor.py cleans non-printable ASCII, normalizes unicode bullets and dashes, standardizes whitespace, and tokenizes text while preserving technical symbols.
Step 5	Entity Parsing & Section Boundary Segmentation	resume_parser.py extracts Name, Email, and Phone using regex & heuristics, then slices the resume into standard sections: Education, Experience, Projects, and Certifications.
Step 6	Controlled Skill Extraction (Resume & JD)	skill_extractor.py evaluates both the cleaned resume text and the job description against the skills.json dictionary using boundary matching and alias normalization.
Step 7	Set-Theoretic Skill Comparison	similarity.py performs set algebra: calculates intersection (Matching Skills), set difference (Missing Skills & Additional Skills), and computes the Skill Overlap Percentage.
Step 8	Structured JSON Return & Dashboard Presentation	FastAPI returns a JSON payload. The React frontend populates the 4 Top Summary Cards, the 3-column color-coded skill chips, and the Candidate Profile card.